

Build a Mini Weather Station with the BME280 & LattePanda Mu

A complete wiring and programming guide for the Lite Carrier (DFR1142) running Windows 11 — measure temperature, humidity and barometric pressure in under an hour.

3.3 V I2C

Python + winsdk

No Arduino needed

Classroom ready

Module: LattePanda Mu (N100 / N305) · Carrier: DFR1142 Lite

OS: Windows 11 · Python 3.10–3.12 · Sensor: Bosch BME280, address 0x76 / 0x77

Sources: Bosch BME280 datasheet (BST-BME280-DS001-23) · LattePanda documentation (docs.lattepanda.com) · driver files: LattePanda-Mu GitHub

Contents

1	What you will build and learn	3
2	Parts and tools	4
3	Meet the BME280	5
4	Wiring the sensor	6
5	Prepare Windows 11 (one time, ~10 minutes)	8
6	Step-by-step build	9
7	How the code works	14
8	Troubleshooting	15
9	Take it further	16
A	References	17

1 What you will build and learn

This guide takes you from an empty workbench to a working weather sensor that reports live **temperature**, **humidity** and **barometric pressure** directly from your LattePanda Mu — the full Windows computer built into a credit-card-size module.

Learning goals

- Understand what the **I2C bus** is: a two-wire, addressed serial bus, and how to find devices on it.
- Read a **datasheet**: registers, chip id and calibration values.
- Wire a real sensor with only four cables, at 3.3 V logic.
- Write a complete Windows 11 driver in Python using the `winsdk` library.
- Convert raw ADC numbers into physical units — the essence of every sensor project.

Why LattePanda Mu is special: the I2C controllers are native to the Intel PCH and are exposed straight to Windows. No USB-to-I2C adapter, no Arduino — Windows talks to the sensor directly, which makes it a great platform for teaching real embedded concepts at school level.

2 Parts and tools

Item	Specification / part	Notes
LattePanda Mu compute module	N100 or N305	Windows 11 preinstalled
Lite Carrier board	DFR1142	Gravity-4P I2C connector + 40-pin header
BME280 breakout module	e.g. DFRobot SEN0236, or any 3.3 V module	check for a 5 V / 3.3 V power pad and choose 3.3 V
4 jumper wires (or Gravity PH2.0 cable)	female-to-female dupont	red, black, yellow, blue
Optional	logic analyser	see the I2C waveforms in class

3.3 V only! Every header pin on the Lite Carrier runs at 3.3 V logic. Do not connect 5 V peripherals, and never power a 5 V-only module. Power the BME280 from the carrier's 3.3 V rail.

3 Meet the BME280

The BME280 measures temperature (−40...+85 °C), humidity (0...100 %RH) and barometric pressure (300...1100 hPa) on one chip. It speaks I2C, offers a selectable address, and always answers with a fixed chip id — the two identity checks you will use to prove the wiring works.

Property	Value	Used for
I2C 7-bit address	0x76 (SDO to GND) / 0x77 (SDO to 3.3 V)	addressing the sensor
Chip id register 0xD0	0x60	confirming it is a BME280
Data registers	0xF7 pressure, 0xFA temp, 0xFD humidity	reading raw measurements
Calibration registers	0x88–0xA1, 0xE1–0xE7	factory trim values per chip
Control registers	0xF2 ctrl_hum, 0xF4 ctrl_meas	starting a measurement
Pull-up resistors	2.2 kΩ to 3.3 V, inside the Mu	no external pull-ups needed

Because the address is selectable, you can even hang **two BME280s on the same SDA/SCL pair** (one with SDO at GND, one with SDO at 3.3 V) — a nice experiment for comparing two spots in the classroom.

4 Wiring the sensor

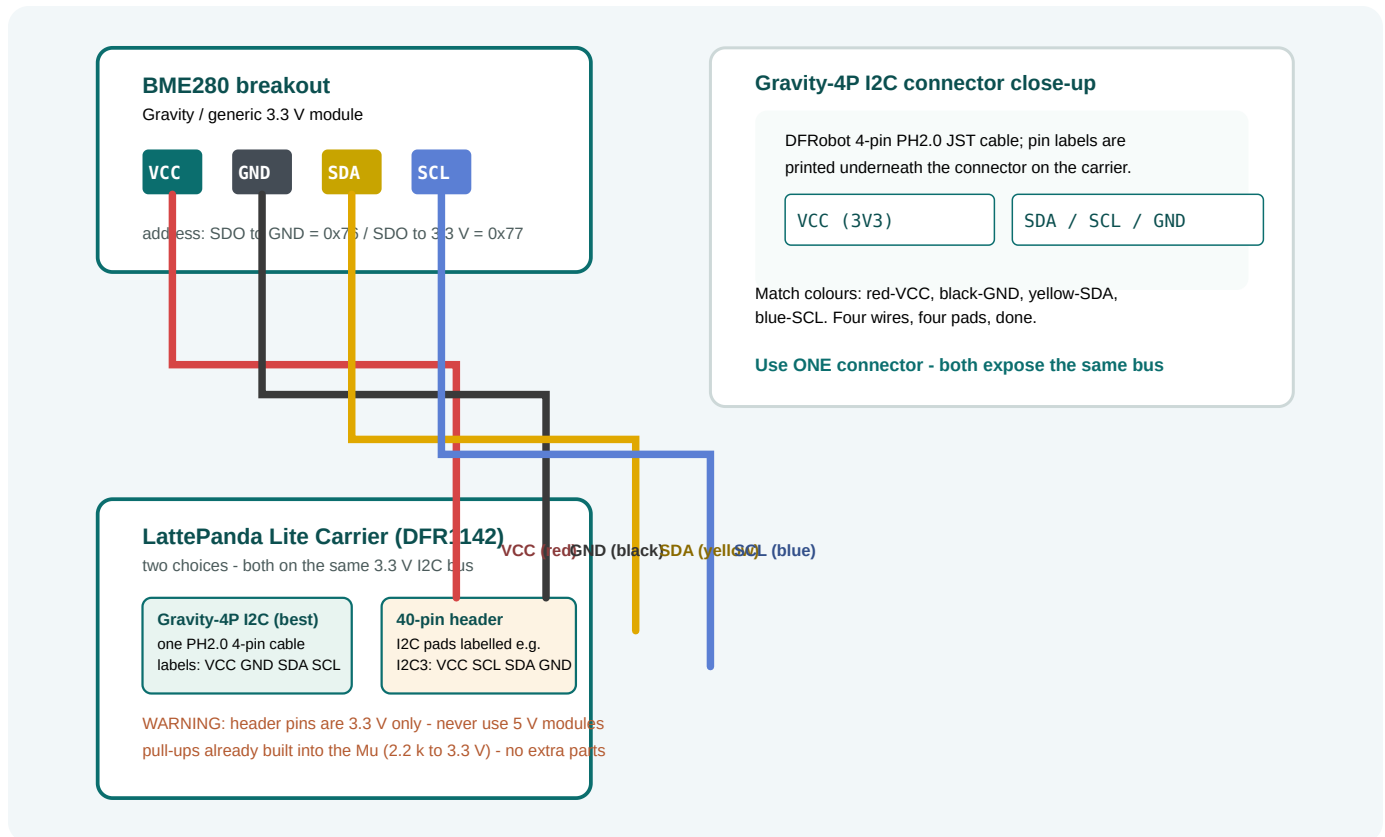


Fig. 1 - BME280 to the Lite Carrier: four wires, two connector choices.

Carrier pin (either connector)	BME280 module pin	Wire colour
VCC (3V3)	VCC / VIN	red
GND	GND	black
SDA	SDA	yellow
SCL	SCL	blue

Where to plug in

- **Option A — the Gravity-4P I2C connector (recommended):** one PH2.0 four-pin cable, plug and go. The pin labels (VCC, GND, SDA, SCL) are printed underneath the connector on the carrier.
- **Option B — the 40-pin GPIO header:** several I2C pad groups are exposed, labelled on the silk screen (e.g. I2C3). The pin order is not Raspberry-Pi style — always follow the printed labels. Same 3.3 V bus as Option A.

Wiring rules for students

1. Disconnect power before plugging or unplugging anything.
2. Double-check SDA and SCL before powering on.
3. Never force a cable into the JST connector — it clicks in one way only.

5 Prepare Windows 11 (one time)

The I2C ports only become visible to Windows after three one-time items. Do these once, then every later project boots straight into Python.

1. Update the BIOS

I2C control under Windows requires BIOS `S70NC1R200-8G-B` (build date 2026/06/03) or newer (16 G / SATA variants included); older versions do not expose the ports. Press `Del` at boot to check the BIOS version.

2. Install the Intel drivers

From LattePanda's official Mu repository on GitHub, install the Chipset driver and the Serial IO driver (the Serial IO driver exposes the I2C host controllers to Windows). After installation, Device Manager shows "Intel(R) Serial IO I2C Host Controller" entries.

3. Install Python and winsdk

1. Install Python **3.10, 3.11 or 3.12** (64-bit) from python.org — the `winsdk` library does not support Python 3.13+.
2. Install the I2C library: `pip install winsdk`

The Mu exposes four I2C controllers to Windows, named **I2C2, I2C3, I2C4, I2C5**. The label printed on the carrier may not match the OS name — the scanner in 6.2 identifies the right one for you.

6 Step-by-step build

Five short steps; 6.1 and 6.3 are the only hardware actions.

6.1 Wire the sensor

Power down, connect the four wires from Chapter 4 (VCC, GND, SDA, SCL), power up, and wait for Windows.

6.2 Scan for the sensor

Save and run the scanner below. It probes every address 0x03–0x77 on the bus and prints the ones that answer. Your BME280 should appear as `0x76`.

```
# i2c_scan.py - find I2C devices on a LattePanda Mu port (Windows 11)
import asyncio, inspect
from winsdk.windows.devices.enumeration import DeviceInformation
from winsdk.windows.devices.i2c import (I2cBusSpeed, I2cConnectionSettings,
                                         I2cDevice, I2cSharingMode)
from winsdk import Array

def run(x):
    # run a WinRT async API synchronously
    if not inspect.isawaitable(x):
        return x
    async def _a(x):
        return await x
    return asyncio.run(_a(x))

BUS = "I2C2" # try I2C2, I2C3, I2C4, I2C5

selector = I2cDevice.get_device_selector(BUS)
devices = run(DeviceInformation.find_all_async(selector, Array(str, [])))
if not devices:
    raise SystemExit(f"No I2C controller '{BUS}' found - check BIOS>=S70NC1R200-8G-B")
dev_id = devices[0].id

found = []
for addr in range(0x03, 0x78):
    try:
        s = I2cConnectionSettings(addr)
        s.bus_speed = I2cBusSpeed.STANDARD_MODE
        s.sharing_mode = I2cSharingMode.SHARED
        probe = run(I2cDevice.from_id_async(dev_id, s))
        if probe is None:
            continue
        probe.read(bytearray(1)) # device ACKs only if it exists
        found.append(addr); probe.close()
    except Exception:
        pass
print(", ".join(f"0x{a:02X}" for a in found) if found else "No device found")
```

Try `I2C2` first; if it says “No device found”, change `BUS` to `I2C3`, `I2C4`, then `I2C5`. Whichever bus prints `0x76` is your port.

6.3 Optional: set the address with SDO

If your module answers at `0x77` instead, its SDO pin is tied high — either leave it and set `ADDR = 0x77`, or short SDO to GND to return to `0x76`.

6.4 Run the weather station

Save this program as `bme280_mu.py`, set `BUS` to your port, and run it:

```

# bme280_mu.py - read temperature, pressure and humidity
# LattePanda Mu + Lite Carrier, Windows 11, Python 3.10-3.12
import asyncio, inspect, time
from winsdk.windows.devices.enumeration import DeviceInformation
from winsdk.windows.devices.i2c import (I2cBusSpeed, I2cConnectionSettings,
                                         I2cDevice, I2cSharingMode)

from winsdk import Array

BUS = "I2C2"          # port where the scanner found your sensor
ADDR = 0x76           # BME280 7-bit address (SDO to GND); 0x77 if SDO to 3V3
REG_ID = 0xD0         # chip-id register
REG_CTRL_HUM = 0xF2
REG_CTRL_MEAS = 0xF4
REG_CALIB_88 = 0x88   # 26 bytes: dig_T1..dig_P9, dig_H1
REG_CALIB_E1 = 0xE1   # 7 bytes: dig_H2..dig_H6
REG_DATA = 0xF7       # 8 bytes: pressure(3) + temperature(3) + humidity(2)

def run(x):            # run a WinRT async API synchronously
    if not inspect.isawaitable(x):
        return x
    async def _a(x):
        return await x
    return asyncio.run(_a(x))

def s12(v):
    return v - 4096 if v & 0x800 else v

def s16(b, i):
    v = b[i] | (b[i+1] << 8)
    return v - 65536 if v & 0x8000 else v

def parse_calib(b26, b7):
    return {
        "T1": b26[0] | (b26[1] << 8), "T2": s16(b26, 2), "T3": s16(b26, 4),
        "P1": b26[6] | (b26[7] << 8), "P2": s16(b26, 8), "P3": s16(b26, 10),
        "P4": s16(b26, 12), "P5": s16(b26, 14), "P6": s16(b26, 16),
        "P7": s16(b26, 18), "P8": s16(b26, 20), "P9": s16(b26, 22),
        "H1": b26[25], "H2": s16(b7, 0), "H3": b7[2],
        "H4": s12((b7[3] << 4) | (b7[4] & 0x0F)),
        "H5": s12(((b7[4] >> 4) & 0x0F) | (b7[5] << 4)),
        "H6": b7[6] - 256 if b7[6] & 0x80 else b7[6],
    }

def compensate(d, adcT, adcP, adcH):
    var1 = (adcT/16384.0 - d["T1"]/1024.0) * d["T2"]
    var2 = ((adcT/131072.0 - d["T1"]/8192.0) ** 2) * d["T3"]
    t_fine = var1 + var2
    temp_c = t_fine / 5120.0
    var1 = t_fine/2.0 - 64000.0
    var2 = var1*var1*d["P6"]/32768.0 + var1*d["P5"]*2.0
    var2 = var2/4.0 + d["P4"]*65536.0
    var1 = (d["P3"]*var1*var1/524288.0 + d["P2"]*var1)/524288.0
    var1 = (1.0 + var1/32768.0) * d["P1"]
    if var1 == 0.0:
        press_pa = 0.0
    else:
        p = 1048576.0 - adcP
        p = (p - var2/4096.0)*6250.0/var1

```

```

    press_pa = p + (d["P9"]*p*p/2147483648.0 + p*d["P8"]/32768.0 + d["P7"])/16.0
    var = t_fine - 76800.0
    var = (adcH - (d["H4"]*64.0 + d["H5"]/16384.0*var)) * (d["H2"]/65536.0) * \
        (1.0 + (d["H6"]/67108864.0*var) * (1.0 + (d["H3"]/67108864.0*var)))
    hum = max(0.0, min(100.0, var * (1.0 - d["H1"]*var/524288.0)))
    return temp_c, press_pa, hum

# --- connect to the bus -----
selector = I2cDevice.get_device_selector(BUS)
devices = run(DeviceInformation.find_all_async(selector, Array(str, [])))
if not devices:
    raise SystemExit(f"No I2C controller '{BUS}' found")

settings = I2cConnectionSettings(ADDR)
settings.bus_speed = I2cBusSpeed.STANDARD_MODE      # 100 kHz
settings.sharing_mode = I2cSharingMode.SHARED
sensor = run(I2cDevice.from_id_async(devices[0].id, settings))
if sensor is None:
    raise SystemExit(f"BME280 not found at 0x{ADDR:02X} on {BUS}")

try:
    # prove the wiring: the chip id register must read 0x60
    chip_id = bytearray(1)
    sensor.write_read(bytes([REG_ID]), chip_id)
    if chip_id[0] != 0x60:
        raise SystemExit(f"Unexpected chip id 0x{chip_id[0]:02X} (want 0x60)")
    print(f"BME280 confirmed (chip id 0x60) on {BUS} @ 0x{ADDR:02X}")

    # read the factory calibration values
    c88 = bytearray(26); sensor.write_read(bytes([REG_CALIB_88]), c88)
    ce1 = bytearray(7);  sensor.write_read(bytes([REG_CALIB_E1]), ce1)
    d = parse_calib(c88, ce1)

    while True:
        sensor.write(bytes([REG_CTRL_HUM, 0x01]))      # humidity oversampling x1
        sensor.write(bytes([REG_CTRL_MEAS, 0x25]))    # temp x1, press x1, forced mode
        time.sleep(0.05)                             # conversion time
        raw = bytearray(8)
        sensor.write_read(bytes([REG_DATA]), raw)
        adcP = (raw[0] << 12) | (raw[1] << 4) | (raw[2] >> 4)
        adcT = (raw[3] << 12) | (raw[4] << 4) | (raw[5] >> 4)
        adcH = (raw[6] << 8) | raw[7]
        temp_c, press_pa, hum = compensate(d, adcT, adcP, adcH)
        print(f"T {temp_c:6.2f} C | P {press_pa/100.0:7.2f} hPa | RH {hum:5.1f} %")
        time.sleep(2)

finally:
    sensor.close()

```

6.5 Observe

You should see one line every two seconds, similar to this real logged output (calibration values differ per chip, so your numbers will vary slightly):

```
BME280 confirmed (chip id 0x60) on I2C2 @ 0x76  
T 26.01 C | P 1040.85 hPa | RH 50.7 %  
T 26.01 C | P 1040.85 hPa | RH 50.7 %  
T 26.01 C | P 1040.85 hPa | RH 50.7 %
```

Cup your hands around the sensor and breathe on it: humidity and temperature move within a second or two. That instant response is the moment students remember.

7 How the code works

What the program does, layer by layer

1. The bus name. `I2cDevice.get_device_selector(BUS)` locates the physical controller registered by the Intel Serial IO driver; `from_id_async` opens it. This is the WinRT I2C API — the same one available to C# and C++.

2. The handshake. Every I2C transaction is: send the register address, then read the reply; `write_read` packs both into one bus transaction. Reading the chip id (0xD0 = 0x60) proves wiring and address before anything else.

3. Calibration. Every BME280 ships with unique factory-trim numbers stored on-chip (Chapter 3, Table 2). Raw ADC counts mean nothing without them. The `compensate()` function converts raw counts into °C, hPa and %RH exactly as Bosch specifies in datasheet section 4.2.3. The float form used here was verified in this project against a compiled C transcription of the datasheet's signed-integer formulas on 1,200 realistic calibration/raw vectors — max deviation 0.008 °C, 0.7 Pa and 0.01 %RH.

Registers used

Register	Bytes read	Purpose
0xD0	1	chip id (must be 0x60)
0xF4 (ctrl_meas)	1	oversampling + mode (0x25 = temp x1, pressure x1, forced)
0xF2 (ctrl_hum)	1	humidity oversampling (0x01 = x1)
0x88...0xA1	26	dig_T1..T3, P1..P9, H1 calibration
0xE1...0xE7	7	dig_H2..H6 calibration
0xF7...0xFE	8	readout: pressure 20-bit, temperature 20-bit, humidity 16-bit

Why the sensor "disappears" if you wire SDA/SCL reversed

I2C is a shared bus: every device listens for its own address. If SDA and SCL are swapped, the chip can never receive its address byte, so nothing answers — the scanner returns empty. Reversing the two data lines is the most common beginner mistake and the easiest to fix.

8 Troubleshooting

Symptom	Most likely cause	Fix
“No I2C controller found”	Serial IO driver missing, or BIOS older than S70NC1R200-8G-B	install the drivers from the LattePanda-Mu GitHub; update the BIOS (Chapter 5)
Scanner runs but silently prints nothing	wrong bus name, or SDA/SCL reversed	try I2C3–I2C5; re-check SDA/SCL against the silk-screen labels
“Unexpected chip id”	the device at that address is not a BME280 (e.g. a BMP280)	check the module label; verify SDO for 0x76 vs 0x77
Temperature fine but humidity reads ~0	the module is actually a BMP280 (no humidity element)	chip id 0x58 = BMP280; use a BMP280-specific guide
Values never change	forced-mode measurement was not re-triggered	the loop writes 0xF4 = 0x25 before every read — keep that line
<code>pip install winsdk</code> fails	Python 3.13+ has no winsdk wheel	install Python 3.10–3.12

Measurement mode note: the example uses *forced mode* (bits 1:0 of 0xF4 = 01): the chip sleeps, takes one set of readings when told, then sleeps again — ideal for battery-friendly classroom projects. The 50 ms wait covers the conversion time at x1 oversampling; raise it to 80 ms if you increase oversampling.

9 Take it further

- **Log to CSV** — append temperature, pressure and humidity with timestamps and open the file in Excel; chart the whole day.
- **Add a second BME280** at 0x77 — compare two ends of the classroom.
- **Show it on an SSD1306 OLED** (address 0x3C) — same `winsdk` bus, same `write_read` pattern; students see readings on a screen.
- **Pressure to altitude** — apply the barometric formula and walk the sensor upstairs to watch altitude change.
- **Go wireless** — the Mu runs full Windows, so any USB or Wi-Fi pipeline works; the I2C lesson stays identical.

A References

- Bosch Sensortec, *BME280 datasheet* BST-BME280-DS001-23 — compensation formulas in section 4.2.3 and calibration storage in Table 16.
- LattePanda documentation — DFR1142 Lite Carrier:
docs.lattepanda.com/content/mu_edition/lite_carrier/
- LattePanda documentation — Windows GPIO programming with Python and winsdk:
docs.lattepanda.com/content/mu_edition/gpio_ctl_windows_python/
- LattePanda-Mu GitHub (drivers and design files): github.com/LattePandaTeam/LattePanda-Mu
- Verification: the compensation code in Chapter 6 is the exact code tested during the production of this guide against a compiled C transcription of the Bosch datasheet formulas (1,200 vectors: calibration round-trip exact; max deviation 0.008 °C / 0.7 Pa / 0.01 %RH across the operating range).